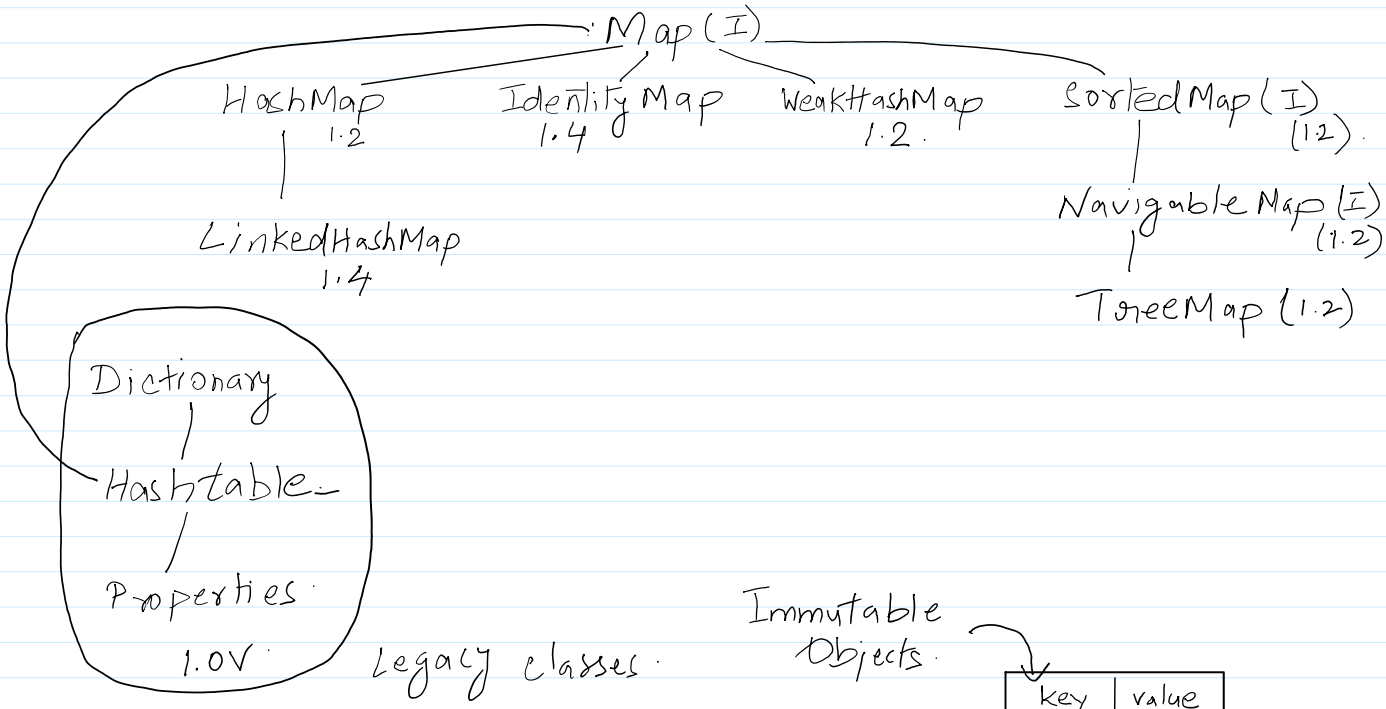
Map(I)

04 May 2025 09:50



Immutable Objects.

Entry

key	value
101	Akash
102	Aman
103	Sahil
104	Ayush
105	Pawan

Map interface is not a child interface of Collection.
If we want to represent a group of objects as key - value pair, then we should go for map.
Both keys and values are objects only.
Duplicate keys are not allowed.
But values can be duplicated.
Each key - value pair is called entry.
Hence, map is considered as a collection of entry objects.

Map (I) methods.

1. Object put (Object key, Object value)

old value.

non-existent key

```
m.put(101, "Arjun");
m.put(102, "Ravi");
m.put(102, "Amay")
```

null

null

"Ravi"

m →

101	"Arjun"
102	"Amay" → "Ravi"
103	"Raj"
104	"Tanvi"

To add one key-value pair to the map object. If the key is already present, then old value will be replaced with new value and returns old value.

Object putAll (Map m)

Object get (Object key) ← returns the value associated with specified key.

Object put(...)

Object get(Object key) ← returns the value associated with specified key.

Object remove(Object key) ← removes the entry associated with specified key.

boolean containsKey(Object key)

boolean containsValue(Object value).

boolean isEmpty()

Set keySet()
Collection values()
Set entrySet()
} Collection views of Map.

Entry (I) // inner interface of Map(I).

```
interface Map {
    interface Entry {
        Object get()
        Object getValue()
        Object setValue(Object newObject)
    }
}
```

} Entry specific method. we can apply only on Entry objects.

A Map is a group of key - value pair. And each key - value pair is called an Entry interface. Hence, Map is considered as a Collection of Entry objects. without existing Map object, there is no chance of existing entry object. Hence, Entry interface is defined inside Map interface.

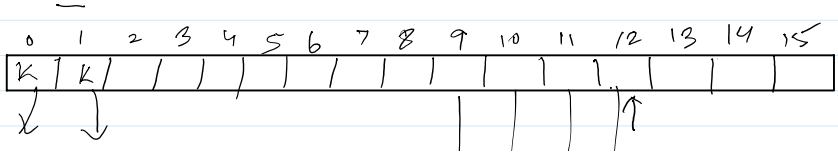
Modifier and Type	Method	Description
void	clear()	Removes all of the mappings from this map (optional operation).
default V	compute(K key, BiFunction<? super K,? super V,? extends V> remappingFunction)	Attempts to compute a mapping for the specified key and its current mapped value (or null if there is no current mapping).
default V	computeIfAbsent(K key, Function<? super K,? extends V> mappingFunction)	If the specified key is not already associated with a value (or is mapped to null), attempts to compute its value using the given mapping function and enters it into this map unless null.
default V	computeIfPresent(K key, BiFunction<? super K,? super V,? extends V> remappingFunction)	If the value for the specified key is present and non-null, attempts to compute a new mapping given the key and its current mapped value.
boolean	containsKey(Object key)	Returns true if this map contains a mapping for the specified key.
boolean	containsValue(Object value)	Returns true if this map maps one or more keys to the specified value.
Set<Map.Entry<K,V>>	entrySet()	Returns a Set view of the mappings contained in this map.
boolean	equals(Object o)	Compares the specified object with this map for equality.
default void	forEach(BiConsumer<? super K,? super V> action)	Performs the given action for each entry in this map until all entries have been processed or the action throws an exception.
V	get(Object key)	Returns the value to which the specified key is mapped, or null if this map contains no mapping for the key.
default V	getOrDefault(Object key, V defaultValue)	Returns the value to which the specified key is mapped, or defaultValue if this map contains no mapping for the key.
int	hashCode()	Returns the hash code value for this map.
boolean	isEmpty()	Returns true if this map contains no key-value mappings.
Set<K>	keySet()	Returns a Set view of the keys contained in this map.
default V	merge(K key, V value, BiFunction<? super V,? super V,? extends V> remappingFunction)	If the specified key is not already associated with a value or is associated with null, associates it with the given non-null value.
V	put(K key, V value)	Associates the specified value with the specified key in this map (optional operation).
void	putAll(Map<? extends K,? extends V> m)	Copies all of the mappings from the specified map to this map (optional operation).
default V	putIfAbsent(K key, V value)	If the specified key is not already associated with a value (or is mapped to null) associates it with the given value and returns null, else returns the current value.
V	remove(Object key)	Removes the mapping for a key from this map if it is present (optional operation).
default boolean	remove(Object key, Object value)	Removes the entry for the specified key only if it is currently mapped to the specified value.
default V	replace(K key, V value)	Replaces the entry for the specified key only if it is currently mapped to some value.
default boolean	replace(K key, V oldValue, V newValue)	Replaces the entry for the specified key only if currently mapped to the specified value.
default void	replaceAll(BiFunction<? super K,? super V,? extends V> function)	Replaces each entry's value with the result of invoking the given function on that entry until all entries have been processed or the function throws an exception.
int	size()	Returns the number of key-value mappings in this map.
Collection<V>	values()	Returns a Collection view of the values contained in this map.

HashMap

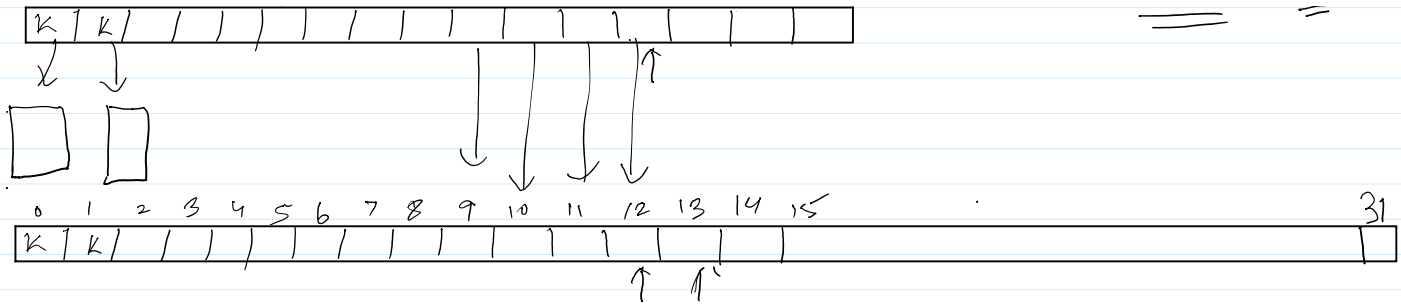
The underlying data structure is HashTable
Insertion order is not preserved and it is based on Hash code of key.
Duplicate keys are not allowed, but values can be duplicated.
Heterogenous objects are allowed for both key and value.
'null' is allowed for key(only once), 'null' is allowed for values(any number of times)
HashMap implements Map<k><v>, Serializable, Cloneable interfaces but not RandomAccess interface.
HashMap is the best choice, if our frequent operation is search operation.

Constructors

HashMap m = new HashMap(); => Creates an empty HashMap Object with default initial capacity 16. And default fill ratio or load factor is 0.75.



$16 \times 0.75 = 12$



HashMap m = new HashMap(int initialCapacity); => Creates empty HashMap Object with specified initial capacity and default fill ratio or load factor is 0.75.

HashMap m = new HashMap(int initialCapacity, float fillRatio); ==> [0.0<fillRatio<=1.0]

HashMap m = new HashMap(Map m); ==> Any Map type object will be converted into HashMap object.

Official Documentation:
public class HashMap<K,V>
extends AbstractMap<K,V>
implements Map<K,V>, Cloneable, Serializable
Hash table based implementation of the Map interface. This implementation provides all of the optional map operations, and permits null values and the null key. (The HashMap class is roughly equivalent to Hashtable, except that it is unsynchronized and permits nulls.) This class makes no guarantees as to the order of the map; in particular, it does not guarantee that the order will remain constant over time.
This implementation provides constant-time performance for the basic operations (get and put), assuming the hash function disperses the elements properly among the buckets. Iteration over collection views requires time proportional to the "capacity" of the HashMap instance (the number of buckets) plus its size (the number of key-value mappings). Thus, it's very important not to set the initial capacity too high (or the load factor too low) if iteration performance is important.
An instance of HashMap has two parameters that affect its performance: *initial capacity* and *load factor*. The *capacity* is the number of buckets in the hash table, and the initial capacity is simply the capacity at the time the hash table is created. The *load factor* is a measure of how full the hash table is allowed to get before its capacity is automatically increased. When the number of entries in the hash table exceeds the product of the load factor and the current capacity, the hash table is *rehashed* (that is, internal data structures are rebuilt) so that the hash table has approximately twice the number of buckets.
As a general rule, the default load factor (.75) offers a good tradeoff between time and space costs. Higher values decrease the space overhead but increase the lookup cost (reflected in most of the operations of the HashMap class, including get and put). The expected number of entries in the map and its load factor should be taken into account when setting its initial capacity, so as to minimize the number of rehash operations. If the initial capacity is greater than the maximum number of entries divided by the load factor, no rehash operations will ever occur.
If many mappings are to be stored in a HashMap instance, creating it with a sufficiently large capacity will allow the mappings to be stored more efficiently than letting it perform automatic rehashing as needed to grow the table. Note that using many keys with the same hashCode() is a sure way to slow down performance of any hash table. To ameliorate impact, when keys are Comparable, this class may use comparison order among keys to help break ties.
Note that this implementation is not synchronized. If multiple threads access a hash map concurrently, and at least one of the threads modifies the map structurally, it *must* be synchronized externally. (A structural modification is any operation that adds or deletes one or more mappings; merely changing the value associated with a key that an instance already contains is not a structural modification.) This is typically accomplished by synchronizing on some object that naturally encapsulates the map. If no such object exists, the map should be "wrapped" using the Collections.synchronizedMap method. This is best done at creation time, to prevent accidental unsynchronized access to the map:

```
Map m = Collections.synchronizedMap(new HashMap(...));
```

The iterators returned by all of this class's "collection view methods" are *fail-fast*: if the map is structurally modified at any time after the iterator is created, in any way except through the iterator's own remove method, the iterator will throw a ConcurrentModificationException. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined

time in the future.
Note that the fail-fast behavior of an iterator cannot be guaranteed as it is, generally speaking, impossible to make any hard guarantees in the presence of unsynchronized concurrent modification. Fail-fast iterators throw ConcurrentModificationException on a best-effort basis. Therefore, it would be wrong to write a program that depended on this exception for its correctness: *the fail-fast behavior of iterators should be used only to detect bugs.*
This class is a member of the Java Collections Framework.

Since:

1.2

From <https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/HashMap.html>

```
import java.util.Collection;
import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;
import java.util.Set;
public class HashMapDemo {
    public static void main(String[] args) {
        System.out.println();
        HashMap m = new HashMap();
        m.put("Ajay", 1000);
        m.put("Harsh", 700);
        m.put("Subha", 600);
        m.put("Gargi", 800);
        m.put("Raj", 250);
        m.put("Uday", 1200);
        System.out.println(m);

        Set s = m.keySet();
        System.out.println("Set of Keys: "+ s);
        Collection c = m.values();
        System.out.println("Collection of values: " + c);
        Set s1 = m.entrySet();
        System.out.println("Enter set: " + s1 );
        Iterator itr = s1.iterator();
        while(itr.hasNext()){
            Map.Entry m1 = (Map.Entry)itr.next();
            System.out.println(m1.getKey() + "===>" + m1.getValue());
            String key = (String)m1.getKey();
            if(key.equals("Raj")){
                m1.setValue(10000);
            }
        }
        System.out.println(m);
    }
}
```

Output:

```
{Subha=600, Harsh=700, Raj=250, Gargi=800, Ajay=1000, Uday=1200}
Set of Keys: [Subha, Harsh, Raj, Gargi, Ajay, Uday]
Collection of values: [600, 700, 250, 800, 1000, 1200]
Enter set: [Subha=600, Harsh=700, Raj=250, Gargi=800, Ajay=1000, Uday=1200]
Subha===>600
Harsh===>700
Raj===>250
Gargi===>800
Ajay===>1000
Uday===>1200
{Subha=600, Harsh=700, Raj=10000, Gargi=800, Ajay=1000, Uday=1200}
```

How to get synchronized version of HashMap Object?

HashMap m = new HashMap(); → not-synchronized

How to get synchronized version of HashMap Object?

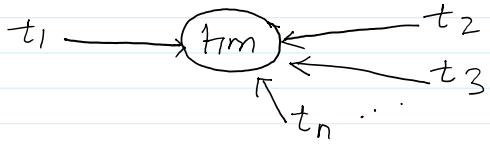
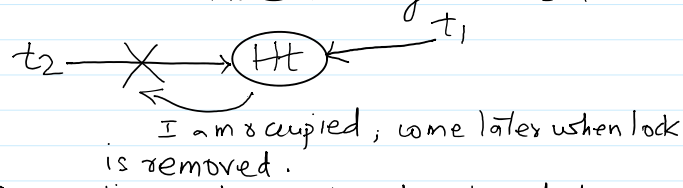
```
HashMap m = new HashMap();

Map m1 = Collections.synchronizedMap(m);
```

not-synchronized

synchronized.

By default, HashMap is non-synchronized, but we can get synchronized version of HashMap by using synchronizedMap() method of Collections class.

Difference between HashMap and HashTable

HashMap	HashTable
1. Not Synchronized: Every method in HashMap is not synchronized.  At a time multiple threads are allowed to operate on HashMap Object. and hence it is not Thread safe.	1. Synchronized: Every method in the HashTable is synchronized.  At a time only one thread is allowed to operate on HashTable and hence it is Thread safe.
2. Relatively performance is high because thread are not required to wait to operate on HashMap object.	2. Relatively performance is low because threads are required to wait to operate on HashTable object.
3. 'null' is allowed for key (only once). 'null' is allowed for value (multiple time).	3. 'null' such type of concept is not available. if given then NullPointerExceptiom will occur.
4. It is not a legacy, it is introduce in 1.2V.	4. It is a legacy, introduce in 1.0v.

LinkedHashMap

It is a child class of HashMap.
It is exactly same as HashMap. (including method and constructor, except the following differences.)

HashMap	LinkedHashMap
The underlying data structure is hash table.	Underline data structure is a combination of LinkedList and HashTable (hybrid data structure)
Insertion order is not preserved and it is based on hash code of keys.	Insertion order is preserved.
1.2V	1.4V

```
import java.util.Collection;
import java.util.LinkedHashMap;
import java.util.Iterator;
import java.util.Map;
```

```java
import java.util.Set;
public class LinkedHashMapDemo {
    public static void main(String[] args) {
        System.out.println();
        LinkedHashMap m = new LinkedHashMap();
        m.put("Ajay", 1000);
        m.put("Harsh", 700);
        m.put("Subha", 600);
        m.put("Gargi", 800);
        m.put("Raj", 250);
        m.put("Uday", 1200);
        System.out.println(m);

        Set s = m.keySet();
        System.out.println("Set of Keys: "+ s);
        Collection c = m.values();
        System.out.println("Collection of values: " + c);
        Set s1 = m.entrySet();
        System.out.println("Enter set: " + s1 );
        Iterator itr = s1.iterator();
        while(itr.hasNext()){
            Map.Entry m1 = (Map.Entry)itr.next();
            System.out.println(m1.getKey() + "===>" + m1.getValue());
            String key = (String)m1.getKey();
            if(key.equals("Raj")){
                m1.setValue(10000);
            }
        }
        System.out.println(m);
    }
}
```

→ we changed HashMap with LinkedHashMap

Output:

→ Insertion order is preserved.

```
{Ajay=1000, Harsh=700, Subha=600, Gargi=800, Raj=250, Uday=1200}
Set of Keys: [Ajay, Harsh, Subha, Gargi, Raj, Uday]
Collection of values: [1000, 700, 600, 800, 250, 1200]
Enter set: [Ajay=1000, Harsh=700, Subha=600, Gargi=800, Raj=250, Uday=1200]
Ajay===>1000
Harsh===>700
Subha===>600
Gargi===>800
Raj===>250
Uday===>1200
{Ajay=1000, Harsh=700, Subha=600, Gargi=800, Raj=10000, Uday=1200}
```

HashMap ↑ LinkedHashMap (LL + HT)

HashSet ↑ LinkedHashSet (LL+HT)

LinkedHashSet and LinkedHashMap are commonly used for developing cache based application.

public class LinkedHashMap<K,V> extends HashMap<K,V> implements Map<K,V>

Hash table and linked list implementation of the Map interface, with predictable iteration order. This implementation differs from HashMap in that it maintains a doubly-linked list running through all of its entries. This linked list defines the iteration ordering, which is normally the order in which keys were inserted into the map (*insertion-order*). Note that insertion order is not affected if a key is *re-inserted* into the map. (A key k is reinserted into a map m if m.put(k, v) is invoked when m.containsKey(k) would return true immediately prior to the invocation.)
This implementation spares its clients from the unspecified, generally chaotic ordering provided by HashMap (and Hashtable),

without incurring the increased cost associated with TreeMap. It can be used to produce a copy of a map that has the same order as the original, regardless of the original map's implementation:

```
void foo(Map<String, Integer> m) {
  Map<String, Integer> copy = new LinkedHashMap<>(m);
  ...
}
```

This technique is particularly useful if a module takes a map on input, copies it, and later returns results whose order is determined by that of the copy. (Clients generally appreciate having things returned in the same order they were presented.)
A special constructor is provided to create a linked hash map whose order of iteration is the order in which its entries were last accessed, from least-recently accessed to most-recently (*access-order*). This kind of map is well-suited to building LRU caches.
Invoking the put, putIfAbsent, get, getOrDefault, compute, computeIfAbsent, computeIfPresent, or merge methods results in an access to the corresponding entry (assuming it exists after the invocation completes). The replace methods only result in an access of the entry if the value is replaced. The putAll method generates one entry access for each mapping in the specified map, in the order that key-value mappings are provided by the specified map's entry set iterator. *No other methods generate entry accesses.* In particular, operations on collection-views do *not* affect the order of iteration of the backing map.
The removeEldestEntry(Map.Entry) method may be overridden to impose a policy for removing stale mappings automatically when new mappings are added to the map.
This class provides all of the optional Map operations, and permits null elements. Like HashMap, it provides constant-time performance for the basic operations (add, contains and remove), assuming the hash function disperses elements properly among the buckets. Performance is likely to be just slightly below that of HashMap, due to the added expense of maintaining the linked list, with one exception: Iteration over the collection-views of a LinkedHashMap requires time proportional to the *size* of the map, regardless of its capacity. Iteration over a HashMap is likely to be more expensive, requiring time proportional to its *capacity*.
A linked hash map has two parameters that affect its performance: *initial capacity* and *load factor*. They are defined precisely as for HashMap. Note, however, that the penalty for choosing an excessively high value for initial capacity is less severe for this class than for HashMap, as iteration times for this class are unaffected by capacity.
Note that this implementation is not synchronized. If multiple threads access a linked hash map concurrently, and at least one of the threads modifies the map structurally, it *must* be synchronized externally. This is typically accomplished by synchronizing on some object that naturally encapsulates the map. If no such object exists, the map should be "wrapped" using the Collections.synchronizedMap method. This is best done at creation time, to prevent accidental unsynchronized access to the map:

```
Map m = Collections.synchronizedMap(new LinkedHashMap(...));
```

A structural modification is any operation that adds or deletes one or more mappings or, in the case of access-ordered linked hash maps, affects iteration order. In insertion-ordered linked hash maps, merely changing the value associated with a key that is already contained in the map is not a structural modification. **In access-ordered linked hash maps, merely querying the map with get is a structural modification.**)
The iterators returned by the iterator method of the collections returned by all of this class's collection view methods are *fail-fast*: if the map is structurally modified at any time after the iterator is created, in any way except through the iterator's own remove method, the iterator will throw a ConcurrentModificationException. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined time in the future.
Note that the fail-fast behavior of an iterator cannot be guaranteed as it is, generally speaking, impossible to make any hard guarantees in the presence of unsynchronized concurrent modification. Fail-fast iterators throw ConcurrentModificationException on a best-effort basis. Therefore, it would be wrong to write a program that depended on this exception for its correctness: *the fail-fast behavior of iterators should be used only to detect bugs.*
The spliterators returned by the spliterator method of the collections returned by all of this class's collection view methods are *late-binding*, *fail-fast*, and additionally report Spliterator.ORDERED.
This class is a member of the Java Collections Framework.

Implementation Note:

The spliterators returned by the spliterator method of the collections returned by all of this class's collection view methods are created from the iterators of the corresponding collections.

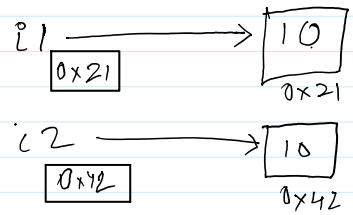
Since:

1.4

From <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/LinkedHashMap.html >

Constructor	Description
`LinkedHashMap()`	Constructs an empty insertion-ordered `LinkedHashMap` instance with the default initial capacity (16) and load factor (0.75).
`LinkedHashMap(int initialCapacity)`	Constructs an empty insertion-ordered `LinkedHashMap` instance with the specified initial capacity and a default load factor (0.75).
`LinkedHashMap(int initialCapacity, float loadFactor)`	Constructs an empty insertion-ordered `LinkedHashMap` instance with the specified initial capacity and load factor.
`LinkedHashMap(int initialCapacity, float loadFactor, boolean accessOrder)`	Constructs an empty `LinkedHashMap` instance with the specified initial capacity, load factor and ordering mode.
`LinkedHashMap(Map<? extends K,? extends V> m)`	Constructs an insertion-ordered `LinkedHashMap` instance with the same mappings as the specified map.

IdentityHashMap



HashMap m = new HashMap();
Integer i1 = new Integer(10);
Integer i2 = new Integer(10);

m.put(i1, "Pawan");
m.put(i2, "Kalyan");

.equals() method is called to identifies the keys. i1.equals(i2)

System.out.println(m); → {10, "Kalyan"}

```
import java.util.HashMap;
public class IdentityHashMapDiff {
    public static void main(String[] args) {
        HashMap m = new HashMap();
        Integer i1 = 10;
        Integer i2 = 10;
        m.put(i1, "Pawan");
        m.put(i2, "Kalyan");
        System.out.println(m);
    }
}
```

Output:

```
{10=Kalyan}
```

```
import java.util.HashMap;
import java.util.IdentityHashMap;
public class IdentityHashMapDiff {
    public static void main(String[] args) {
        IdentityHashMap m = new IdentityHashMap();
        Integer i1 = new Integer(10);
        Integer i2 = new Integer(10);
        m.put(i1, "Pawan");
        m.put(i2, "Kalyan");
        System.out.println(m);
    }
}
```

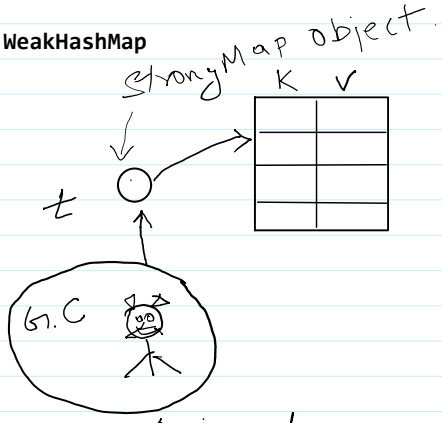
Output:

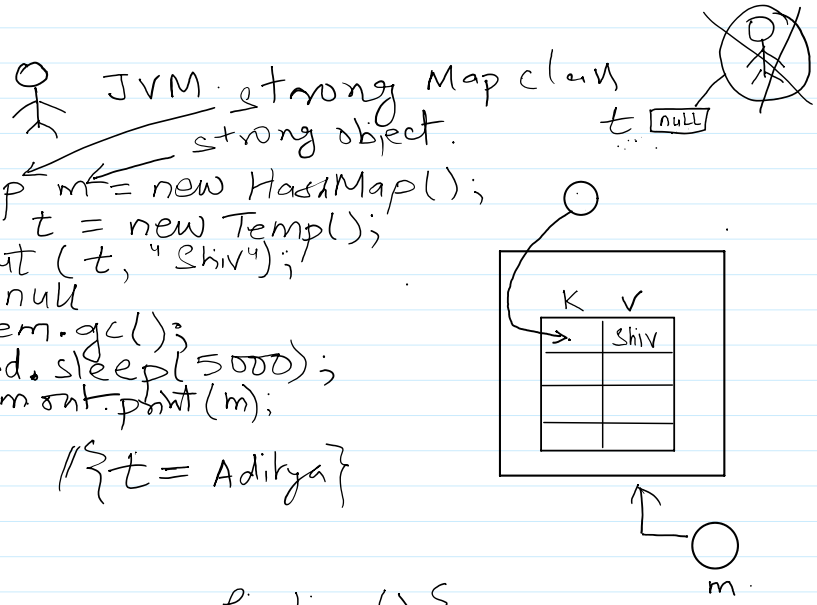
```
{10=Pawan, 10=Kalyan}
```

IdentityHashMap is exactly same as HashMap(including methods and constructors.)
In case of HashMap JVM, will use .equals() method to identify, duplicate keys, which is made for content

comparison.
But in the case of IdentityHashMap, JVM will use '==' operator to identify, duplicate keys, which is meant for reference comparison (address comparison).

WeakHashMap






```
class Temp{
    public String toString() {
        return "temp";
    }
    public void finalize(){
        System.out.println("finalize method call");
    }
}
```

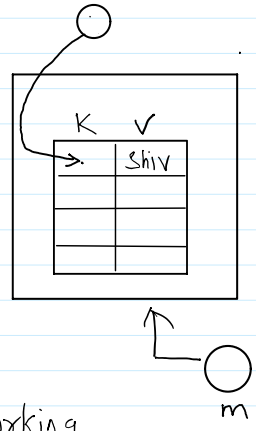
```
finalize() {
    clean/update.
}
```

← called by Garbage Collector before clean up activities.

```
import java.util.HashMap;
public class WeakHashMapDemo {
    public static void main(String[] args) throws InterruptedException {
        HashMap h = new HashMap();
        Temp t = new Temp();
        h.put(t, "Shiv");
        t = null;
        System.gc();
        Thread.sleep(5000);
        System.out.println(h);
    }
}
```

Stronger than gc()
Why? because reference is present in the memory.

even the reference is sleeping.



Output:

{Temp=Shiv}

```
public class Temp {
    @Override
    public String toString(){
        return "Temp";
    }
    public void finalize(){
        System.out.println("Finalize method called");
    }
}
```

method call indicates Garbage Collector is working

```java
import java.util.WeakHashMap;
public class WeakHashMapDemo {
    public static void main(String[] args) throws InterruptedException {
        WeakHashMap h = new WeakHashMap();
        Temp t = new Temp();
        h.put(t, "Shiv");
        t = null;
        System.gc();
        Thread.sleep(5000);
        System.out.println(h);
    }
}
```

} ← 't' is deleted by gc() because Map object is weak.

Output:

Finalize method called

{}

WeekHashMap is exactly same as HashMap, except the following differences.

In case of **HashMap**, even though object does not have any reference, it is eligible for garbage collection. If it is associated with HashMap, that is hash map dominates garbage collector. So, gc() will not be able to remove the HashMap object from the memory.

But in the case of **WeakHashMap**, if the object does not contain any reference, it is eligible for garbage collection, even though object associated with WeakHashMap that is garbage collector dominates the WeakHashMap. So the object and WeakhashMap object is deleted from the memory.